

Advantages and Disadvantages of Indexes in SQL Server

In this article, I am going to discuss the **Advantages and Disadvantages of Indexes in SQL Server** with examples. Please read our previous article where we discussed [Index in Group By Clause in SQL Server](#). This is one of the frequently asked interview questions in SQL Server and I am sure at the end of this article, you will be in a better position to answer the Advantages and Disadvantages of Indexes in SQL Server question.

Advantages of Indexes in SQL Server:

To understand the advantages of Indexes in SQL Server, we are going to use the following Employee table.

ID	FirstName	LastName	Salary	Gender	City	Dept
1	Pranaya	Rout	4500	Male	Mumbai	IT
2	Anurag	Mohanty	2500	Male	Delhi	IT
3	Priyanka	Dewgaan	5500	Female	Hyderabad	HR
4	Sambit	Satapathy	3000	Male	Hyderabad	IT
5	Preety	Tiwary	6500	Female	Mumbai	HR
6	Tarun	Mallick	4000	Male	Delhi	IT
7	Hina	Sharma	5000	Female	Delhi	HR

Please use the below SQL Script to create and populate the Employee table with the required sample data.

```
-- Create Employee table
CREATE TABLE Employee
(
    ID INT PRIMARY KEY,
    FirstName VARCHAR(30),
    LastName VARCHAR(30),
    Salary INT,
    Gender VARCHAR(30),
    City VARCHAR(30),
    Dept VARCHAR(30)
)
-- Populate the Employee table with some test data
INSERT INTO Employee VALUES (1,'Pranaya', 'Rout', 4500, 'Male', 'Mumbai', 'IT')
INSERT INTO Employee VALUES (2,'Anurag','Mohanty', 2500, 'Male', 'Delhi', 'IT')
INSERT INTO Employee VALUES (3,'Priyanka','Dewgaan', 5500, 'Female', 'Hyderabad', 'HR')
INSERT INTO Employee VALUES (4,'Sambit', 'Satapathy', 3000, 'Male', 'Hyderabad', 'IT')
INSERT INTO Employee VALUES (5,'Preety', 'Tiwary', 6500, 'Female', 'Mumbai', 'HR')
INSERT INTO Employee VALUES (6,'Tarun', 'Mallick', 4000, 'Male', 'Delhi', 'IT')
INSERT INTO Employee VALUES (7,'Hina', 'Sharma', 5000, 'Female', 'Delhi', 'HR')
```

Create a Non-Clustered Index on the Salary column of the Employee table

CREATE NONCLUSTERED INDEX IX_Employee_Salary ON Employee (Salary)

Now the Index table stores the data something similar to the below image.

Salary	Row Address
2500	Row Address
3000	Row Address
4000	Row Address
4500	Row Address
5000	Row Address
5500	Row Address
6500	Row Address

Let's discuss the advantages of using Indexes in SQL Server

Searching For Records

The most common use for an index in SQL Server is to find a record or set of records matching a WHERE clause condition. Please consider the following SQL Query.

SELECT * FROM Employee WHERE Salary > 3000 AND Salary < 6000

The above SQL Select query gets benefits from the index that we created on the Salary column. This is because the salaries are sorted in ascending order in the index. From the index, it's easy to identify the records where the salary is between 3000 and 6000, and using the row address the corresponding records from the table can be fetched quickly.

In SQL Server, not only the SELECT statement but also the DELETE and UPDATE statements can get benefit from the index. Please have a look at the following DELETE and UPDATE statements

DELETE FROM Employee where Salary = 5500

UPDATE Employee SET Salary = 8000 WHERE Salary = 6500

To update or delete a row, the SQL server needs to first find that row, and the index can help in searching and finding that specific row quickly which ultimately improves the performance of the query.

Sorting Records

When we ask for a sorted result set, first, the database will try to find an index and try to avoid sorting the results during the execution of the query. We can control the sorting of a result set by specifying a field, or fields, in an ORDER BY clause, with the sort order as ASC (ascending) or DESC (descending). For example, the following query returns all the employees sorted by Salary:

SELECT * FROM Employee ORDER BY Salary

As we have an index on the Salary column of the Employee table, the Salaries are already sorted. Here, the database engine simply scans the index from the first entry to the last entry and retrieves the rows in sorted order. This avoids the sorting of rows during query execution, which can significantly improve the processing

time. The same index works equally well with the following query simply by scanning the index in reverse order.

```
SELECT * FROM Employee ORDER BY Salary DESC
```

Grouping Records

We can use a GROUP BY clause to group the records and aggregate values. For example, counting the number of employees for each department. To process a query with a GROUP BY clause, the database will often sort the results on the columns included in the GROUP BY clause. As of now, we do not have any index on the department column. So let's first create an index on the department column.

```
CREATE NONCLUSTERED INDEX IX_Employee_Dept ON Employee (Dept)
```

The following query counts the number of employees at each department by grouping together records with the same department value.

```
SELECT Dept, COUNT(Dept) AS EmployeeCount FROM Employee GROUP BY Dept
```

Here, to group the Employees with the same department, the query engine can use the index IX_Employee_Dept to retrieve the already sorted departments. Since the matching departments are present in consecutive index entries, it is to count the total number of employees at each Department quickly.

Maintaining a Unique Column

The Columns which require unique values (such as primary key columns) must have a unique index applied. There are several methods available in SQL Server to create a unique index. Marking a column as a primary key will automatically create a unique index on the column as we already discussed in our previous article. Let's create a unique index on the FirstName column.

```
CREATE UNIQUE INDEX IDX_Employee_FirstName On Employee (FirstName)
```

The above SQL command will not allow any duplicate values in the FirstName column, and an index is the best tool for the database to use to enforce this rule. Each time an application adds or modifies a row in the table, the database needs to search all existing records to ensure none of the values in the new data duplicates existing values.

Disadvantages of Indexes:

As we already discussed, indexes provide an outstanding performance benefit to searches, but there is also a downside to indexing. Let's discuss some of those drawbacks.

Additional Disk space:

The Clustered index in SQL Server does not require any additional disk space. But each and every non-clustered index requires additional disk space as it is stored separately from the table. The amount of space required will depend on the size of the table, and the number and types of columns used in the index.

Generally, the Disk space is cheap enough to trade for application performance, particularly when a database serves a large number of users. To see the space required for a table, use the sp_spaceused system stored procedure as shown below.

EXEC sp_spaceused Employee

Provide the table name to the system-stored procedure sp_spaceused which will return the amount of space used by the data and all indexes associated with the table as shown below.

name	rows	reserved	data	index_size	unused
Employee	6	64 KB	8 KB	56 KB	0 KB

According to the above output, the table data uses 8 kilobytes, while the table indexes use 56 kilobytes. The ratio of index size to table size can vary greatly, depending on the columns, data types, and the number of indexes on a table.

Insert Update and delete statement become Slow:

Another drawback of using indexes in SQL Server is the performance implication on data modification statements. When any DML statement such as Insert, Update and Delete modifies data in a table, the database needs to update all of the indexes where data has changed. As we already discussed the indexes can help to search and locate the rows that we want to modify, however, too many indexes to update can actually hurt the performance of data modifications. This leads to a delicate balancing act when tuning the database for performance.

A Disadvantage to Clustered Indexes

If we update a record and change the value of an indexed column in a clustered index, the database might need to move the entire row into a new position to keep the rows in sorted order. This behavior essentially turns an update query into a DELETE followed by an INSERT, with an obvious decrease in performance. A table's clustered index can often be found on the primary key or a foreign key column because key values generally do not change once a record is inserted into the database.

In the next article, I am going to discuss the [Built-in String Functions in SQL Server](#) with examples. Here, In this article, I try to explain the **Advantages and Disadvantages of Indexes in SQL Server** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about the Advantages and Disadvantages of Indexes in the SQL Server article.